

2.7 math.h

The math header defines several mathematic functions.

Macros:

`HUGE_VAL`

Functions:

```
acos();  
asin();  
atan();  
atan2();  
ceil();  
cos();  
cosh();  
exp();  
fabs();  
floor();  
fmod();  
frexp();  
ldexp();  
log();  
log10();  
modf();  
pow();  
sin();  
sinh();  
sqrt();  
tan();  
tanh();
```

2.7.1 Error Conditions

All math.h functions handle errors similarly.

In the case that the argument passed to the function exceeds the range of that function, then the variable `errno` is set to `EDOM`. The value that the function returns is implementation specific.

In the case that the value being returned is too large to be represented in a double, then the function returns the macro `HUGE_VAL`, and sets the variable `errno` to `ERANGE` to represent an overflow. If the value is too small to be represented in a double, then the function returns zero. In this case whether or not `errno` is set to `ERANGE` is implementation specific.

`errno`, `EDOM`, and `ERANGE` are defined in the `errno.h` header.

Note that in all cases when it is stated that there is no range limit, it is implied that the value is limited by the minimum and maximum values of type double.

2.7.2 Trigonometric Functions

2.7.2.1 acos

Declaration:

```
double acos(double x);
```

Returns the arc cosine of x in radians.

Range:

The value x must be within the range of -1 to +1 (inclusive). The returned value is in the range of 0 to π (inclusive).

2.7.2.2 asin

Declaration:

```
double asin(double x);
```

Returns the arc sine of x in radians.

Range:

The value of x must be within the range of -1 to +1 (inclusive). The returned value is in the range of $-\pi/2$ to $+\pi/2$ (inclusive).

2.7.2.3 atan

Declaration:

```
double atan(double x);
```

Returns the arc tangent of x in radians.

Range:

The value of x has no range. The returned value is in the range of $-\pi/2$ to $+\pi/2$ (inclusive).

2.7.2.4 atan2

Declaration:

```
double atan2(double y, double x);
```

Returns the arc tangent in radians of y/x based on the signs of both values to determine the correct quadrant.

Range:

Both y and x cannot be zero. The returned value is in the range of $-\pi/2$ to $+\pi/2$ (inclusive).

2.7.2.5 cos

Declaration:

```
double cos(double x);
```

Returns the cosine of a radian angle x .

Range:

The value of x has no range. The returned value is in the range of -1 to $+1$ (inclusive).

2.7.2.6 cosh

Declaration:

```
double cosh(double x);
```

Returns the hyperbolic cosine of x .

Range:

There is no range limit on the argument or return value.

2.7.2.7 sin

Declaration:

```
double sin(double x);
```

Returns the sine of a radian angle x .

Range:

The value of x has no range. The returned value is in the range of -1 to $+1$ (inclusive).

2.7.2.8 sinh

Declaration:

```
double sinh(double x);
```

Returns the hyperbolic sine of x .

Range:

There is no range limit on the argument or return value.

2.7.2.9 tan

Declaration:

```
double tan(double x);
```

Returns the tangent of a radian angle x .

Range:

There is no range limit on the argument or return value.

2.7.2.10 tanh

Declaration:

```
double tanh(double x);
```

Returns the hyperbolic tangent of x .

Range:

The value of x has no range. The returned value is in the range of -1 to +1 (inclusive).

2.7.3 Exponential, Logarithmic, and Power Functions

2.7.3.1 exp

Declaration:

```
double exp(double x);
```

Returns the value of e raised to the x th power.

Range:

There is no range limit on the argument or return value.

2.7.3.2 frexp

Declaration:

```
double frexp(double x, int *exponent);
```

The floating-point number x is broken up into a mantissa and exponent.
The returned value is the mantissa and the integer pointed to by *exponent* is the exponent.
The resultant value is $x = \text{mantissa} * 2^{\text{exponent}}$.

Range:

The mantissa is in the range of .5 (inclusive) to 1 (exclusive).

2.7.3.3 ldexp

Declaration:

```
double ldexp(double x, int exponent);
```

Returns x multiplied by 2 raised to the power of *exponent*.
 $x * 2^{\text{exponent}}$

Range:

There is no range limit on the argument or return value.

2.7.3.4 log

Declaration:

```
double log(double x);
```

Returns the natural logarithm (base-e logarithm) of x .

Range:

There is no range limit on the argument or return value.

2.7.3.5 log10

Declaration:

```
double log10(double x);
```

Returns the common logarithm (base-10 logarithm) of x .

Range:

There is no range limit on the argument or return value.

2.7.3.6 modf

Declaration:

```
double modf(double x, double *integer);
```

Breaks the floating-point number x into integer and fraction components. The returned value is the fraction component (part after the decimal), and sets *integer* to the integer component.

Range:

There is no range limit on the argument or return value.

2.7.3.7 pow

Declaration:

```
double pow(double x, double y);
```

Returns x raised to the power of y .

Range:

x cannot be negative if y is a fractional value. x cannot be zero if y is less than or equal to zero.

2.7.3.8 sqrt

Declaration:

```
double sqrt(double x);
```

Returns the square root of x .

Range:

The argument cannot be negative. The returned value is always positive.

2.7.4 Other Math Functions

2.7.4.1 ceil

Declaration:

```
double ceil(double x);
```

Returns the smallest integer value greater than or equal to x .

Range:

There is no range limit on the argument or return value.

2.7.4.2 fabs

Declaration:

```
double fabs(double x);
```

Returns the absolute value of x (a negative value becomes positive, positive value is unchanged).

Range:

There is no range limit on the argument. The return value is always positive.

2.7.4.3 floor

Declaration:

```
double floor(double x);
```

Returns the largest integer value less than or equal to x .

Range:

There is no range limit on the argument or return value.

2.7.4.4 fmod

Declaration:

```
double fmod(double x, double y);
```

Returns the remainder of x divided by y .

Range:

There is no range limit on the return value. If y is zero, then either a range error will occur or the function will return zero (implementation-defined).